Exercises: Employment Status Patterns in the `nlsy97_emp_hed.parquet` Dataset

In the following, let us use the NLSY97 Monthly Employment and College Enrollment Status data file (available on Moodle): `nlsy97_emp_hed.parquet`.

I. Loading the data

1. Let us load the necessary packages:

```
# external (third-party) packages
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

import math
```

2. Let us define the path to the project directory (I name it `PROJECT_DIR` this time) and the path to the directory in which our data file is located (I name it `DATA_DIR` this time):

```
# Your path will almost certainly differ from the one below.
PROJECT_DIR = r'C:\Users\PC_DS_ECON_5\Desktop\data-analytics-python-wise2627'

# The relative path could also differ from the one below.
DATA_DIR = PROJECT_DIR + '/local/data'

print(PROJECT_DIR)
print(DATA_DIR)
```

3. Let us read in the `nlsy97_emp_hed.parquet` file using `pandas.read_parquet()`:

```
df = pd.read_parquet(DATA_DIR + '/' + 'nlsy97_emp_hed.parquet')
```

(Reminder: To read the file, one needs a Parquet engine, such as the one provided by the `pyarrow` package. The package can be installed in the usual way with `pip install pyarrow` after activating the virtual environment.)

II. Employment patterns around the onset of the COVID-19 pandemic

The NLSY97 is the National Longitudinal Survey of Youth 1997. It follows a specific cohort of individuals over time, from their teenage years onwards. The individuals in the sample were born between 1980 and 1984.

In the data frame, the unit of observation is `person_id` × `year_month`. The `emp_category` variable contains information about the employment status of respondents in a given month.

In what follows, the objective is to see whether, and if so how, the onset of the COVID-19 pandemic was reflected in employment patterns in the NLSY97.

Let us create a copy of our data frame and refer to it as `df2`:

```
df2 = df.copy()
```

From now on, let us work with `df2` instead of `df`.

(a) Let us restrict our attention to observations from the years 2019 and 2020. Let us use the `.loc[]` accessor together with the `between()` and `eq()` methods to select observations from these two years for the so-called `Cross-sectional` sample (which is designed to be representative) by running the following code:

```
df2 = (
    df2
        .loc[df2['year'].between(2019, 2020) &
             df2['sample'].eq('Cross-sectional'), :]
        .reset_index(drop=True)
        .copy()
)
```

Does it make sense to keep observations from 2019 as well? Why or why not?

(b) Let us keep observations for individuals for whom the employment status `emp_category` is never missing over the selected two-year period of analysis.

Doing so involves two steps.

In the first step, we need to count the non-missing observations for `emp_category` for each individual.

- Let us group observations by individual using the `groupby()` method and select the column `emp_category`.
- Let us then use the `transform()` method with a `lambda` function that says: for each group of observations (in this particular example, given the grouping by `person_id` and the selection of the column `emp_category`), let us count the number of non-missing values using `notna()` and `sum()`.

Let us assign the result to a new column called `n_emp_category_notna`.

Simply run the following code:

```
df2['n_emp_category_notna'] = (
    df2
       .groupby(by='person_id')['emp_category']
       .transform(lambda group_of_observations: group_of_observations.notna().sum())
)
```

In the second step, we can apply the restriction using `.loc[]` and the `eq()` method.

```
df2 = (
    df2.loc[df2['n_emp_category_notna'].eq(24)].copy()
)
```

Why `eq(24)`*?*

(c) How would you count the number of individuals in the remaining balanced panel?

(d) For every month, let us compute how the individuals are distributed across the five employment statuses. Put differently, for each month, let us compute the relative frequency of each employment status.

There are at least two different ways of doing this.

The first approach uses `groupby()` and `value_counts()`:

```
df3 = (
    df2
        .groupby(by='year_month')['emp_category']
        .value_counts(normalize=True)
        .reset_index()
        .copy()
)
```

We could bring the result into a more compact form, with the employment status categories as columns and the months as rows, using `pivot()`:

```
df3 = (
    df3
        .pivot(
            index='year_month',
            columns='emp_category',
            values='proportion'
        )
        .reset_index()
)
```

The second approach is simpler. We can directly obtain the tabular version using the `crosstab()` function:

```
df3 = pd.crosstab(
    index=df2['year_month'],
    columns=df2['emp_category'],
    normalize='index'
).reset_index()
```

One can verify that the relative frequencies do indeed sum to 1:

```
emp_cat_cols = [
    'employed',
    'intermittently employed',
    'unemployed',
    'non-participant',
    'military']
(
    df3[emp_cat_cols]
        .sum(axis=1)
        .apply(lambda x: math.isclose(x, 1))
        .value_counts()
)
```

(e) Let us plot the content of the cross-tabulation `df3`, i.e. the distribution of individuals across the five employment statuses (or the distribution of employment statuses, if you will). The objective is to obtain a stacked bar plot, with one stack (totalling 1) for each month and the months displayed on the horizontal x-axis.

To be able to plot the dates on the x-axis, we need to convert our `year_month` variable into an appropriate `datetime` variable called `year_month_begin`:

```
print('dtype of year_month:', df3['year_month'].dtype)
df3['year_month_begin'] = (
    df3['year_month'].dt.to_timestamp(freq='D', how='start')
)
print('dtype of year_month_begin:', df3['year_month_begin'].dtype)
```

Let us begin by plotting only the share of employed individuals over time using a bar plot.

To do so, let us use `matplotlib.pyplot` (abbreviated as `plt`). To create a figure with a single subplot, let us use `plt.subplots()`. In the following assignment statement, `plt.subplots()` returns a tuple containing a `Figure` object and a so-called `Axes` object (all this is just `matplotlib` terminology). Let us refer to the figure as `fig` and the axes as `ax`:

```
fig, ax = plt.subplots()
```

Most of the plot attributes can be set via the axes object `ax`.

Let us create our bar plot using `ax.bar()`, with the x-coordinates given by `df3['year_month_begin']` and the bar heights given by `df3['employed']`:

```
fig, ax = plt.subplots()
ax.bar(
    x=df3['year_month_begin'],
    height=df3['employed'],
    width=20
    )
plt.show()
```

Why `width=20`? Because the units on the horizontal axis are days. It therefore seems reasonable to set the width of the bars to 20 days.

If we wish the ticks on the horizontal axis to appear approximately every five months, we can, for example, do the following:

```
fig, ax = plt.subplots()
ax.bar(
    x=df3['year_month_begin'],
    height=df3['employed'],
    width=20
    )
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=5))
plt.show()
```

Stacking works thanks to the `bottom` argument of the `bar()` method. Staying with our example, we could "elevate" the bars such that their tops are located at exactly 1. This could be done by setting `bottom` equal to 1 minus the height. Concretely:

```
df3['1-employed'] = 1 - df3['employed']
fig, ax = plt.subplots()
ax.bar(
    x=df3['year_month_begin'],
    height=df3['employed'],
    bottom=df3['1-employed'],
    width=20
    )
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=5))
ax.set_ylim(bottom=0)
plt.show()
```

And there is nothing to stop us from filling the area below the bars with another layer of bars:

```
df3['1-employed'] = 1 - df3['employed']
fig, ax = plt.subplots()
ax.bar(
    x=df3['year_month_begin'],
    height=df3['employed'],
    bottom=df3['1-employed'],
    width=20
    )
ax.bar(
    x=df3['year_month_begin'],
    height=df3['1-employed'],
    width=20
    )
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=5))
ax.set_ylim(bottom=0)
plt.show()
```

Adding labels and a legend works as well:

```
df3['1-employed'] = 1 - df3['employed']
fig, ax = plt.subplots()
ax.bar(
    x=df3['year_month_begin'],
    height=df3['employed'],
    bottom=df3['1-employed'],
    width=20,
    label='employed'
    )
ax.bar(
    x=df3['year_month_begin'],
    height=df3['1-employed'],
    width=20,
    label='all other employment statuses'
    )
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=5))
ax.set_ylim(bottom=0)
ax.legend()
plt.show()
```

One can, of course, go further and distinguish between all five categories using `for` loops:

```
emp_cat_cols = [
    'employed',
    'intermittently employed',
    'unemployed',
    'non-participant',
    'military'
    ]

df3 = df3[['year_month_begin'] + emp_cat_cols].copy()

for emp_cat in emp_cat_cols:
    df3[emp_cat + '_cumul'] = (
        df3.loc[:, 'employed':emp_cat].sum(axis=1) - df3[emp_cat]
    )

fig = plt.figure()
ax = fig.add_subplot()

colors = ['lightblue', 'purple', 'red', 'pink', 'grey']

for (i, emp_cat) in enumerate(emp_cat_cols):
    ax.bar(
        x=df3['year_month_begin'],
        bottom=df3[emp_cat + '_cumul'],
        height=df3[emp_cat],
```

```
        label=emp_cat,
        color=colors[i],
        width=20
        )

ax.xaxis.set_major_locator(mdates.MonthLocator(interval=5))
ax.legend()
```

III. Employment patterns around the Great Recession

Your turn. How would you construct plots equivalent to those in Section II, showing the distribution of employment statuses, but this time around the Great Recession (2008–2010)?